1. Find the total revenue collected by each shipper.

```
#question no 1
SELECT s.shipper_name,
SUM(oi.quantity * oi.price_each) as total_revenue
from Shippers s
join Orders o on s.shipper_id = o.shipper_id
join Order_Items oi on o.order_id = oi.order_id
group by s.shipper_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

shipper_name	total_revenue
BlueDart Express	149700.00
DTDC Courier	200200.00
Delhivery Logistics	122400.00
Ekart Logistics	123500.00
Gati Express	105400.00
India Post Speed	155100.00
Shadowfax Delivery	100800.00

2. Show the top 5 highest-spending customers along with their total payments

```
#question no 2
SELECT c.name,
sum(oi.quantity * oi.price_each) as total_spent
from Customers c
join Orders o on c.customer_id = o.customer_id
join Order_Items oi on o.order_id = oi.order_id
group by c.name
order by total_spent DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

name	total_spent
Rohit Kumar	152000.00
Megha Kapoor	88500.00
Sneha Sharma	83100.00
Ritika Gupta	77600.00
Tanya Mishra	75200.00

3. Find product categories where the average selling price is greater than 8000.

```
#question no 3
SELECT category,
avg(price) as avg_price
from Products
group by category
having avg(price) > 8000;
```

Result Grid | Filter Rows:

category	avg_price
Electronics	67749.000000

4. Show the total number of orders placed per city, sorted by highest to lowest.

```
#question no 4
SELECT c.city,
count(o.order_id) as total_orders
from Customers c
join Orders o on c.customer_id = o.customer_id
group by c.city
order by total_orders desc;
```

Result Grid | Filter Rows: | Export: | Wrap Ce

city	total_orders
Delhi	10
Chennai	10
Mumbai	6
Bengaluru	6
Ahmedabad	5
Kolkata	5
Lucknow	5
Bhubaneswar	5
Jaipur	5
Noida	5
Indore	5
Hyderabad	4
Pune	4

5. Find suppliers who supply more than 1 product category.

```
#question no 5
SELECT s.supplier_name
from Suppliers s
join Products p on s.supplier_id = p.supplier_id
group by s.supplier_name
having COUNT(distinct p.category) > 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell C

supplier_name

There are no suppliers who supply more than 1 product.

6. Show each order along with the number of items it contains.

```
#question no 6
SELECT o.order_id,
count(oi.order_item_id) as item_count
from Orders o
join Order_Items oi on o.order_id = oi.order_id
group by o.order_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

order_id	item_count
1	1
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	1
13	1
14	1
15	1

7. Find customers who have spent more than the average spending of all customers

```
#question no 7
SELECT name FROM Customers
WHERE customer_id IN (
SELECT o.customer_id
FROM Orders o
JOIN Order_Items oi ON o.order_id = oi.order_id
GROUP BY o.customer_id
HAVING SUM(oi.quantity * oi.price_each) >
(SELECT AVG(total_spent)
FROM (
SELECT SUM(oi2.quantity * oi2.price_each) AS total_spent
                FROM Orders o2
                JOIN Order_Items oi2 ON o2.order_id = oi2.order_id
                GROUP BY o2.customer_id
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name
Rohit Kumar
Sneha Sharma
Amit Patel
Arjun Mehta
Ritika Gupta
Megha Kapoor
Tanya Mishra
Aditya Jain

8. List all products whose price is higher than the average product price in their category.

```
#question no 8
SELECT *
from Products p
where price > (
    select AVG(price)
    from Products
```

Result Grid | Filter Rows: | Edit: | Export/Impo

product_id	product_name	category	price	stock_qty	supplier_id
1	iPhone 15	Electronics	79999.00	50	1
2	Samsung Galaxy S24	Electronics	74999.00	40	2
3	Noise Smartwatch	Wearables	4999.00	100	3
6	Running Shoes	Fashion	2999.00	120	7
NULL	NULL	NULL	NULL	NULL	NULL

9. Show customers who placed at least one order with total_amount greater than 50,000.

```sql
#question no 9
SELECT DISTINCT c.name
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN Order_Items oi ON o.order_id = oi.order_id
GROUP BY c.name, o.order_id
HAVING SUM(oi.quantity * oi.price_each) > 50000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

name
Rohit Kumar

10.Find customers who placed more orders than the average number of orders per customer

```sql
#question no 10
WITH CustomerOrderCounts AS (
SELECT COUNT(order_id) AS order_count
    FROM Orders
    GROUP BY customer_id
)
SELECT customer_id
FROM Orders
GROUP BY customer_id
HAVING COUNT(order_id) > (
SELECT AVG(order_count)
    FROM CustomerOrderCounts
);
```

Result Grid | Filter Rows: | Export:

customer_id
2
7

11. Find the most expensive product(s) in the catalog.

```
#question no 11
SELECT * from Products
where price = (select MAX(price) from Products);
```

Result Grid | Filter Rows: | Edit: | Export

product_id	product_name	category	price	stock_qty	supplier_id
1	iPhone 15	Electronics	79999.00	50	1
NULL	NULL	NULL	NULL	NULL	NULL

12.Rank customers based on their total spending.

```sql
#question no 12
SELECT c.name,
SUM(oi.quantity * oi.price_each) as total_spent,
rank() over (order by SUM(oi.quantity * oi.price_each) DESC) as rank_no
from Customers c
join Orders o on c.customer_id = o.customer_id
join Order_Items oi on o.order_id = oi.order_id
group by c.name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	total_spent	rank_no
Rohit Kumar	152000.00	1
Megha Kapoor	88500.00	2
Sneha Sharma	83100.00	3
Ritika Gupta	77600.00	4
Tanya Mishra	75200.00	5
Aditya Jain	72000.00	6
Amit Patel	70000.00	7
Arjun Mehta	66000.00	8
Ravi Shankar	55500.00	9
Suresh Iyer	48400.00	10
Neha Verma	46400.00	11
Priya Reddy	42600.00	12
Ananya Das	30400.00	13
Karan Singh	24800.00	14
Vikram Joshi	24600.00	15

13.Find cumulative sales amount by order date

```sql
#question no 13
SELECT o.order_date,
SUM(oi.quantity * oi.price_each) as daily_sales,
SUM(SUM(oi.quantity * oi.price_each)) over (order by o.order_date) as cumulative_sales
from Orders o
join Order_Items oi on o.order_id = oi.order_id
group by o.order_date;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_date	daily_sales	cumulative_sales
2024-01-05	30000.00	30000.00
2024-01-06	12000.00	42000.00
2024-01-07	7500.00	49500.00
2024-01-08	25600.00	75100.00
2024-01-09	14000.00	89100.00
2024-01-10	7000.00	96100.00
2024-01-11	10600.00	106700.00
2024-01-12	22000.00	128700.00
2024-01-13	30000.00	158700.00
2024-01-14	25800.00	184500.00
2024-01-15	29000.00	213500.00
2024-01-16	4500.00	218000.00
2024-01-17	8300.00	226300.00
2024-01-18	14000.00	240300.00
2024-01-19	81000.00	321300.00
2024-01-20	5000.00	326300.00
2024-01-21	4800.00	331100.00
2024-01-22	8000.00	339100.00

14.Get each customer’s order count and show their percentage contribution.

```sql
#question no 14
SELECT customer_id,
COUNT(order_id) as order_count,
COUNT(order_id) * 100.0 / SUM(COUNT(order_id)) over () as percentage
from Orders
group by customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	order_count	percentage
1	5	6.66667
2	6	8.00000
3	5	6.66667
4	4	5.33333
5	5	6.66667
6	4	5.33333
7	6	8.00000
8	5	6.66667
9	5	6.66667
10	5	6.66667
11	5	6.66667
12	5	6.66667
13	5	6.66667
14	5	6.66667
15	5	6.66667

15.Show the most recent order per customer.

```sql
#question no 15
SELECT *
FROM Orders
WHERE (customer_id, order_date) IN
(SELECT customer_id, MAX(order_date)
FROM Orders
GROUP BY customer_id);
```

Result Grid | Filter Rows: | Edit:

order_id	customer_id	order_date	payment_id	shipper_id
52	6	2024-02-09	3	7
59	4	2024-02-13	2	5
63	8	2024-02-15	5	2
64	9	2024-02-16	4	7
65	10	2024-02-16	2	6
66	11	2024-02-17	3	1
67	12	2024-02-18	6	5
68	13	2024-02-18	1	4
69	14	2024-02-19	7	3
70	15	2024-02-20	5	2
71	1	2024-02-21	2	7
72	2	2024-02-21	3	6
73	3	2024-02-22	4	1
74	5	2024-02-23	6	5
75	7	2024-02-23	7	4
NULL	NULL	NULL	NULL	NULL

16.List each product with its sales quantity and rank products within each category by total sales.

```
#question no 16
SELECT p.product_name,p.category,
SUM(oi.quantity) AS total_qty,
rank() over (PARTITION BY p.category ORDER BY SUM(oi.quantity) DESC) AS rank_in_categor
from Products p
join Order_Items oi on p.product_id = oi.product_id
group by p.product_name, p.category;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	category	total_qty	rank_in_category
Lenovo Laptop	Electronics	18	1
iPhone 15	Electronics	16	2
Samsung Galaxy S24	Electronics	13	3
Sony Bravia 55 TV	Electronics	10	4
Running Shoes	Fashion	22	1
Kurta Set	Fashion	15	2
Prestige Mixer Grinder	Home Appliances	15	1
Philips Trimmer	Personal Care	11	1
Boat Earbuds	Wearables	13	1
Noise Smartwatch	Wearables	8	2

17.Categorize products as ‘High’, ‘Medium’, or ‘Low’ price using CASE.

```
#question no 17
SELECT product_name,
case
when price >= 70000 then 'High'
when price >= 20000 then 'Medium'
else 'Low'
end as price_category
from Products;
```

Result Grid | Filter Rows: | Export:

product_name	price_category
iPhone 15	High
Samsung Galaxy S24	High
Noise Smartwatch	Low
Boat Earbuds	Low
Kurta Set	Low
Running Shoes	Low
Prestige Mixer Grinder	Low
Sony Bravia 55 TV	Medium
Lenovo Laptop	Medium
Philips Trimmer	Low

18.Use a CTE to find top 3 customers by spending

```
#question no 17
WITH cte as (SELECT c.name,
SUM(oi.quantity * oi.price_each) as total_spent
from Customers c
join Orders o on c.customer_id = o.customer_id
join Order_Items oi on o.order_id = oi.order_id
group by c.name)
SELECT * from cte
ORDER BY total_spent DESC
LIMIT 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	total_spent
Rohit Kumar	152000.00
Megha Kapoor	88500.00
Sneha Sharma	83100.00

19.Use a CTE with CASE to classify customers by loyalty (based on number of orders).

```sql
#question no 19
WITH cte AS (SELECT customer_id,
COUNT(order_id) as orders_count
from Orders group by customer_id)
SELECT customer_id,case
when orders_count >= 5 then 'high'
when orders_count >= 2 then 'medium'
else 'Low' end as loyalty
from cte;
```

Result Grid | Filter Rows: | Export:

customer_id	loyalty
1	high
2	high
3	high
4	medium
5	high
6	medium
7	high
8	high
9	high
10	high
11	high
12	high
13	high
14	high
15	high

20.Find monthly revenue growth percentage compared to the previous month.

```sql
#question no 21
SELECT month, revenue,
((revenue / PREV_REVENUE) - 1) * 100 AS growth_percentage
FROM (SELECT
DATE_FORMAT(o.order_date, '%Y-%m') AS month,
SUM(oi.quantity * oi.price_each) AS revenue,
LAG(SUM(oi.quantity * oi.price_each)) OVER (ORDER BY DATE_FORMAT(o.order_date, '%Y-%m')) AS PREV_REVENUE
FROM Orders o
JOIN Order_Items oi ON o.order_id = oi.order_id
GROUP BY month) t;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

month	revenue	growth_percentage
2024-01	479300.00	NULL
2024-02	477800.00	-0.312956

21.Find top 2 customers per city based on total spending.

```sql
#question no 21
SELECT *
from (SELECT c.city,c.name,
SUM(oi.quantity * oi.price_each) as total_spent,
rank() over (partition by c.city order by SUM(oi.quantity * oi.price_each) DESC) rnk
from Customers c
join Orders o on c.customer_id = o.customer_id
join Order_Items oi on o.order_id = oi.order_id
group by c.city, c.name) t
WHERE rnk <= 2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

city	name	total_spent	rnk
Ahmedabad	Amit Patel	70000.00	1
Bengaluru	Arjun Mehta	66000.00	1
Bhubaneswar	Ananya Das	30400.00	1
Chennai	Suresh Iyer	48400.00	1
Chennai	Karan Singh	24800.00	2
Delhi	Rohit Kumar	152000.00	1
Delhi	Ravi Shankar	55500.00	2
Hyderabad	Priya Reddy	42600.00	1
Indore	Aditya Jain	72000.00	1
Jaipur	Megha Kapoor	88500.00	1
Kolkata	Ritika Gupta	77600.00	1
Lucknow	Vikram Joshi	24600.00	1
Mumbai	Sneha Sharma	83100.00	1
Noida	Tanya Mishra	75200.00	1

22.Find top 3 cities with highest sales revenue including shipper name.

```sql
#question no 22
SELECT c.city,
s.shipper_name,
SUM(oi.quantity * oi.price_each) as revenue
from Customers c
join Orders o on c.customer_id = o.customer_id
join Shippers s on o.shipper_id = s.shipper_id
join Order_Items oi on o.order_id = oi.order_id
group by c.city, s.shipper_name
order by revenue DESC
LIMIT 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

city	shipper_name	revenue
Delhi	DTDC Courier	66000.00
Noida	Delhivery Logistics	56000.00
Jaipur	BlueDart Express	55000.00

23.List all orders with customer name, product name, supplier, and shipper.

```sql
#question no 23
SELECT o.order_id, c.name, p.product_name, s.supplier_name, sh.shipper_name
from Orders o
join Customers c on o.customer_id = c.customer_id
join Order_Items oi on o.order_id = oi.order_id
join Products p on oi.product_id = p.product_id
join Suppliers s on p.supplier_id = s.supplier_id
join Shippers sh on o.shipper_id = sh.shipper_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	name	product_name	supplier_name	shipper_name
1	Rohit Kumar	iPhone 15	Reliance Digital	Delhivery Logistics
12	Megha Kapoor	iPhone 15	Reliance Digital	India Post Speed
21	Ritika Gupta	iPhone 15	Reliance Digital	Ekart Logistics
31	Ritika Gupta	iPhone 15	Reliance Digital	DTDC Courier
41	Rohit Kumar	iPhone 15	Reliance Digital	India Post Speed
50	Ananya Das	iPhone 15	Reliance Digital	DTDC Courier
61	Amit Patel	iPhone 15	Reliance Digital	BlueDart Express
70	Aditya Jain	iPhone 15	Reliance Digital	DTDC Courier
4	Priya Reddy	Samsung Galaxy S24	Croma Electronics	BlueDart Express
11	Suresh Iyer	Samsung Galaxy S24	Croma Electronics	BlueDart Express
22	Rohit Kumar	Samsung Galaxy S24	Croma Electronics	DTDC Courier
32	Arjun Mehta	Samsung Galaxy S24	Croma Electronics	Gati Express
40	Megha Kapoor	Samsung Galaxy S24	Croma Electronics	BlueDart Express
52	Neha Verma	Samsung Galaxy S24	Croma Electronics	Shadowfax Delivery
60	Sneha Sharma	Samsung Galaxy S24	Croma Electronics	India Post Speed
69	Tanya Mishra	Samsung Galaxy S24	Croma Electronics	Delhivery Logistics

24.Show total sales per supplier along with average order value.

```sql
#question no 24
SELECT s.supplier_name,
SUM(oi.quantity * oi.price_each) as total_sales,
avg(oi.quantity * oi.price_each) as avg_order_value
from Suppliers s
join Products p on s.supplier_id = p.supplier_id
join Order_Items oi on p.product_id = oi.product_id
group by s.supplier_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conte

supplier_name	total_sales	avg_order_value
Reliance Digital	240000.00	30000.000000
Croma Electronics	307600.00	20506.666667
Flipkart Seller Hub	96000.00	13714.285714
Amazon India Vendor	104000.00	13000.000000
Tata Cliq	27000.00	3375.000000
Vijay Sales	35000.00	5000.000000
Lifestyle Stores	70500.00	4406.250000
Metro Electronics	77000.00	12833.333333

25.Show product categories that contributed more than 30% of total sales revenue.

```sql
#question no 25
WITH total AS (SELECT SUM(oi.quantity * oi.price_each) as total_rev
from Order_Items oi)
SELECT p.category,
SUM(oi.quantity * oi.price_each) / t.total_rev * 100 as percentage_contribution
from Products p
join Order_Items oi on p.product_id = oi.product_id
cross join total t
group by p.category, t.total_rev
having percentage_contribution > 30;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	percentage_contribution
Electronics	60.871382